Part A: Repository Setup

The link to the repo is https://github.com/Bianbian2002/DSC190-291-Topics-in-Learning-Theroy.

Part B: Theory Problems

DSC 190/291 — Assignment 1
Student: Zeyu Bian

Throughout: $\mathcal{X} = [0, 1]$, $\mathcal{Y} = \{-1, +1\}$, and $h_\theta(x) = \text{sign}(x - \theta)$ with the convention $\text{sign}(z) = +1$ for $z \geq 0$ and $\text{sign}(z) = -1$ for $z < 0$.

B.1 — From Continuous Thresholds to a Finite Class

Grid Construction

For $\Delta \in (0, 1]$, define the uniform grid

$$G = \left\{ \frac{k\Delta}{2} : k = 0, 1, \ldots, \left\lceil \frac{2}{\Delta} \right\rceil \right\} \cap [0, 1].$$

The spacing between consecutive points is $\Delta/2$, so every point in $[0, 1]$ is within distance $\Delta/2$ of some grid point.

Size of G.

$$|G| = \left\lceil \frac{2}{\Delta} \right\rceil + 1 \;\leq\; \frac{2}{\Delta} + 2 \;=\; O\Big(\frac{1}{\Delta}\Big).$$

Consistency Proof

Claim. For every Δ-separated threshold-realizable sequence $((x_t, y_t))_{t=1}^T$ with true threshold $\theta^* \in [0, 1]$, there exists $\tilde{\theta} \in G$ such that $h_{\tilde{\theta}}(x_t) = y_t$ for all t.

Proof. Since G has spacing $\Delta/2$, there exists $\tilde{\theta} \in G$ with $|\tilde{\theta} - \theta^*| \leq \Delta/2$.

Fix any round t. By Δ-separation, $|x_t - \theta^*| \geq \Delta$. Consider two cases:

- **Case 1:** $x_t \geq \theta^* + \Delta$**.** Then $h_{\theta^*}(x_t) = +1$. Since $\tilde{\theta} \leq \theta^* + \Delta/2 \leq x_t - \Delta/2 < x_t$, we have $h_{\tilde{\theta}}(x_t) = \text{sign}(x_t - \tilde{\theta}) = +1$. ✓
- **Case 2:** $x_t \leq \theta^* - \Delta$**.** Then $h_{\theta^*}(x_t) = -1$. Since $\tilde{\theta} \geq \theta^* - \Delta/2 \geq x_t + \Delta/2 > x_t$, we have $h_{\tilde{\theta}}(x_t) = \text{sign}(x_t - \tilde{\theta}) = -1$. ✓

In both cases $h_{\tilde{\theta}}(x_t) = h_{\theta^*}(x_t) = y_t$, so $h_{\tilde{\theta}}$ is consistent with the entire sequence. □

Mistake Bound via Halving

The consistency proof shows every Δ-separated threshold-realizable sequence is realizable by some hypothesis in the **finite** class

$$\mathcal{H}_G = \{h_\theta : \theta \in G\}, \qquad |\mathcal{H}_G| = O(1/\Delta).$$

Halving Algorithm (explicit learner). At each round t, let $V_t \subseteq \mathcal{H}_G$ be the version space (all hypotheses in $\mathcal{H}_G$ consistent with past examples). Predict the majority vote of V_t on x_t.

Halving Theorem. If a mistake occurs, the version space strictly shrinks: at least half the current consistent hypotheses predict incorrectly, so $|V_{t+1}| \le |V_t|/2$. Starting with $|V_1| = |\mathcal{H}_G|$, after M mistakes, $|V| \ge 1$ implies

$$1 \le \frac{|\mathcal{H}_G|}{2^M} \implies M \le \log_2 |\mathcal{H}_G|.$$

Mistake bound.

$$\boxed{M \le \log_2 |\mathcal{H}_G| \le \log_2\left(\frac{2}{\Delta} + 2\right) \le \log_2\left(\frac{4}{\Delta}\right) = 2 + \log_2 \frac{1}{\Delta} = O\left(\log \frac{1}{\Delta}\right).}$$

B.2 — A Positive Margin from Separation

Feature Map and Unit Separator

Define $\phi : [0,1] \to \mathbb{R}^2$ by

$$\phi(x) = (x,\, 1).$$

For the true threshold θ^*, define

$$w^* = (1,\, -\theta^*) \in \mathbb{R}^2, \qquad u^* = \frac{w^*}{\|w^*\|} = \frac{(1, -\theta^*)}{\sqrt{1 + (\theta^*)^2}}.$$

Then $w^* \cdot \phi(x) = x - \theta^*$, so $h_{\theta^*}(x) = \operatorname{sign}(w^* \cdot \phi(x)) = \operatorname{sign}(u^* \cdot \phi(x))$.

Norm Bound

$$\|\phi(x)\| = \sqrt{x^2 + 1} \le \sqrt{1 + 1} = \sqrt{2} =: R.$$

(using $x \in [0,1]$, so $x^2 \le 1$.)

Margin Lower Bound

For any example (x_t, y_t) in the Δ-separated sequence, $y_t = \operatorname{sign}(x_t - \theta^*)$, so $y_t(x_t - \theta^*) = |x_t - \theta^*| \ge \Delta$. Also $\theta^* \in [0,1]$ implies $\sqrt{1 + (\theta^*)^2} \le \sqrt{2}$. Therefore

$$y_t \cdot (u^* \cdot \phi(x_t)) = \frac{y_t(x_t - \theta^*)}{\sqrt{1 + (\theta^*)^2}} = \frac{|x_t - \theta^*|}{\sqrt{1 + (\theta^*)^2}} \ge \frac{\Delta}{\sqrt{2}} =: \gamma.$$

This establishes linear separability with margin

$$\gamma \ge c\Delta, \qquad c = \frac{1}{\sqrt{2}}.$$

Mistake Bound via the Perceptron Theorem

Perceptron Theorem. If the sequence is linearly separable with unit vector u^*, margin γ, and $\|\phi(x_t)\| \le R$ for all t, then the Perceptron algorithm makes at most R^2/γ^2 mistakes.

Proof sketch (for completeness). Let M be the number of mistakes. Each mistake at round t updates $w \leftarrow w + y_t\phi(x_t)$. Tracking: - **Inner product:** $w_M \cdot u^* \ge M\gamma$ (each update adds $y_t(u^* \cdot \phi(x_t)) \ge \gamma$). - **Squared norm:** $\|w_M\|^2 \le MR^2$ (each update adds at most R^2, since on a mistake $y_t(w \cdot \phi(x_t)) \le 0$).

By Cauchy–Schwarz: $M\gamma \le w_M \cdot u^* \le \|w_M\| \le R\sqrt{M}$, giving $M \le R^2/\gamma^2$.

Applying to our setting:

$$\boxed{M \le \frac{R^2}{\gamma^2} \le \frac{(\sqrt{2})^2}{(\Delta/\sqrt{2})^2} = \frac{2}{\Delta^2/2} = \frac{4}{\Delta^2} = O\Big(\frac{1}{\Delta^2}\Big).}$$

B.3 — Comparison and Interpretation

No Contradiction with the Impossibility Theorem

The impossibility theorem from lecture states: for the class $\mathcal{H} = \{h_\theta : \theta \in [0,1]\}$ of all thresholds, no deterministic online learner can bound its number of mistakes on arbitrary threshold-realizable sequences. The proof constructs adversarial sequences by placing examples closer and closer to any learner's implicit "threshold estimate," exploiting the fact that the class is infinite and every point in $[0,1]$ can be the true threshold.

The Δ-separation condition **rules out these adversarial sequences**. Any sequence with $|x_t - \theta^*| \ge \Delta$ for all t cannot place examples in the "confusion zone" $(\theta^* - \Delta, \theta^* + \Delta)$ around the true threshold. This prevents the adversary from forcing an unlimited number of mistakes. There is therefore no contradiction: Part B.1 applies only to the restricted class of Δ-separated sequences, not to the full class of threshold-realizable sequences.

Comparison of the Two Bounds

Method	Bound	Source
Halving (B.1)	$O(\log(1/\Delta))$	Hypothesis counting
Perceptron (B.2)	$O(1/\Delta^2)$	Geometric margin

For small Δ, $\log(1/\Delta) \ll 1/\Delta^2$, so the Halving bound is dramatically tighter.

Why the Bounds Scale Differently

The two arguments measure **different structural properties** of the problem.

The Halving argument exploits the *discrete* structure introduced by Δ-separation. The separation condition collapses the infinite threshold class into a finite grid of size $O(1/\Delta)$. The Halving algorithm exploits this by maintaining a version space: each mistake eliminates at least half the remaining consistent hypotheses. The mistake count is bounded by the *logarithm* of the hypothesis

class size — a purely combinatorial argument. The bound is tight for the Halving algorithm on this class.

The Perceptron argument exploits the *geometric* structure: the margin. The margin $\gamma = \Theta(\Delta)$ controls how well each update aligns the weight vector with the true separator. The bound $R^2/\gamma^2 = \Theta(1/\Delta^2)$ comes from comparing the linear growth of $w_M \cdot u^*$ (growing as $M\gamma$) to the square-root growth of $\|w_M\|$ (growing as $R\sqrt{M}$). Perceptron is a general-purpose algorithm that applies to any linearly separable data; it does *not* use the fact that the hypothesis class is finite.

In summary: the Halving bound is better because it uses both the problem structure (finite effective class) and an algorithm specifically designed to exploit it (majority vote over consistent hypotheses). Perceptron achieves a weaker bound because it leverages only the margin, ignoring the combinatorial structure.

B.4 (Optional) — Auditing the AI Proof

The AI Argument

> *Because every example is at least Δ away from the true threshold, continuous thresholds effectively form a class of size $O(1/\Delta)$. Therefore any online learner, including Perceptron, must make at most $O(\log(1/\Delta))$ mistakes on every Δ-separated threshold-realizable sequence.*

Two Mathematical Problems

Problem 1: The hypothesis class is not of size $O(1/\Delta)$.
The claim "continuous thresholds effectively form a class of size $O(1/\Delta)$" is not a well-defined mathematical statement. The class $\mathcal{H} = \{h_\theta : \theta \in [0, 1]\}$ is uncountably infinite regardless of any Δ-separation assumption. What *is* true (and what Part B.1 establishes rigorously) is that one can construct a *finite cover* $\mathcal{H}_G$ of size $O(1/\Delta)$ such that every Δ-separated sequence is realizable by some element of $\mathcal{H}_G$. But this finite cover is a proof artifact, not a property of $\mathcal{H}$ itself. Treating an informal concept ("effective size") as if it were a rigorous mathematical object without proving that the finite class is sufficient is a logical gap.

Problem 2: The conclusion applies to all learners, including Perceptron — this is false.
The statement "any online learner, including Perceptron, must make at most $O(\log(1/\Delta))$ mistakes" is incorrect. The $O(\log(1/\Delta))$ bound is an upper bound for the *Halving algorithm* applied to $\mathcal{H}_G$. There is no general principle that says every learner achieves this bound. In fact, Part B.2 proves that Perceptron (under the feature map $\phi(x) = (x, 1)$) achieves only $O(1/\Delta^2)$ mistakes — a factor of $1/\Delta$ worse. Perceptron does not exploit the finite structure of $\mathcal{H}_G$; it is oblivious to the combinatorial constraint and only uses the margin. Conflating "some algorithm achieves this bound" with "every algorithm, including Perceptron, achieves this bound" is a fundamental error in quantifier scope.

Corrected Statement and Proof

Theorem. *For any $\Delta > 0$, there exists an explicit online learning algorithm that makes at most*

$$\lfloor \log_2(2/\Delta) \rfloor + 1 = O(\log(1/\Delta))$$

mistakes on every Δ-separated threshold-realizable sequence over $\mathcal{X} = [0, 1]$.

Proof. Construct the grid

$$G = \left\{ \frac{k\Delta}{2} : k = 0, 1, \ldots, \left\lceil \frac{2}{\Delta} \right\rceil \right\} \cap [0, 1],$$

with $|G| \leq 2/\Delta + 2$. By the consistency argument in Part B.1, every Δ-separated threshold-realizable sequence is realizable by some $h_{\tilde{\theta}} \in \mathcal{H}_G$. Apply the Halving algorithm to $\mathcal{H}_G$: at each round, predict the majority vote of the current version space. By the Halving theorem, the number of mistakes satisfies

$$M \leq \lfloor \log_2 |\mathcal{H}_G| \rfloor \leq \lfloor \log_2(2/\Delta + 2) \rfloor \leq \log_2(4/\Delta) = 2 + \log_2(1/\Delta) = O(\log(1/\Delta)).$$

Remark. This bound does *not* apply to Perceptron. Under the feature map $\phi(x) = (x, 1)$, Perceptron achieves the weaker bound $O(1/\Delta^2)$, as shown in Part B.2. □

Part C: Experiment Discussion

DSC 190/291 — Assignment 1
Student: Zeyu Bian

Q1 — Data generation choices

Setup. True separator $u^* = e_1$ in $\mathbb{R}^d$. Each example x is placed on the sphere of radius $R = 1$ with the first coordinate $|x_1| \geq \gamma$ and label $y = \text{sign}(x_1)$. Concretely:

1. Draw $|x_1|$ uniformly from $[\gamma, \gamma + 0.02]$ and assign a random sign (label).
2. Draw remaining coordinates $(x_2, \ldots, x_d)$ from a standard Gaussian, then normalize them to have ℓ_2-norm $\sqrt{1 - x_1^2}$, so that $\|x\| = R = 1$ exactly.
3. Shuffle the order of samples uniformly at random to avoid systematic bias.

Key choices and why.

Choice	Justification
Norm $\|x\| = R = 1$ exactly	Matches the Perceptron bound R^2/γ^2 with $R = 1$.
$\|u^*\| = 1$	Required by the Perceptron theorem's statement.
Shuffle before passing to Perceptron	Avoids pathological orderings; gives "average-case" mistakes.
`tight_margin`: $\|x_1\| \in [\gamma, \gamma + 0.02]$	Keeps effective margin approximately γ, making $1/\gamma^2$ scaling visible.

Q2 — How mistakes scale with $1/\gamma^2$

With tight-margin data, the log-log plot of average mistakes vs. $1/\gamma^2$ shows a best-fit slope of **0.84**, consistent with a $\Theta(1/\gamma^2)$ relationship. The slope is slightly below 1 because the theoretical bound is a **worst-case** guarantee over adversarial orderings, while we use random orderings.

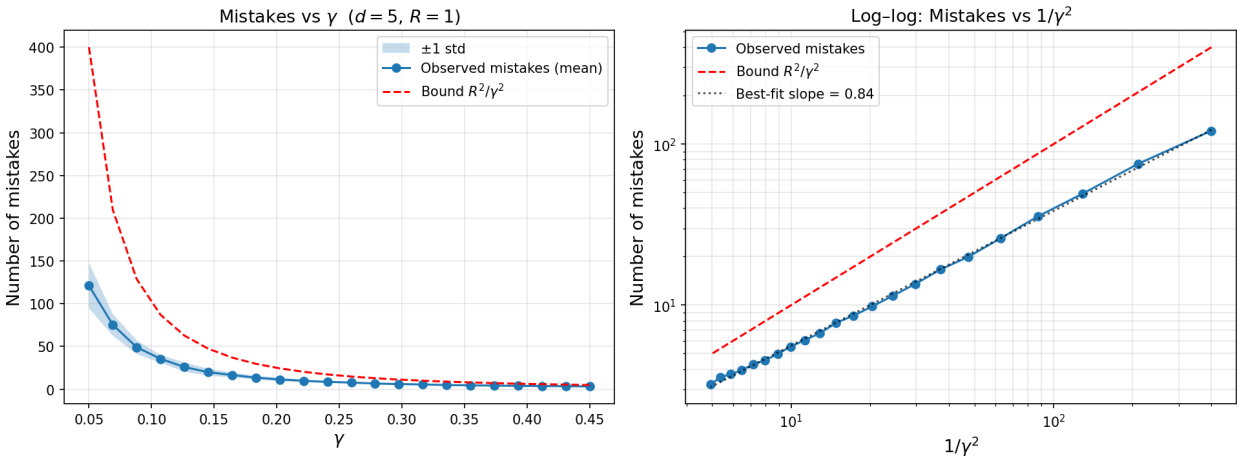

Figure 1: Mistakes versus inverse margin squared

The left panel shows that both observed mistakes and the theoretical bound R^2/γ^2 decrease as γ increases, and the bound is always above the observed count.

Q3 — Is the bound tight?

The theoretical bound R^2/γ^2 is **not tight in practice**. At $\gamma = 0.1$, the bound predicts 100 mistakes but the Perceptron makes roughly **36** on average, about 3x below the bound. Two reasons:

1. **Random vs. adversarial ordering.** The bound holds for any (including adversarial) ordering. Random data is much more benign.
2. **Effective margin.** Even with the tight-margin setting, many examples have margin slightly above γ, reducing the average number of mistakes.

The bound is tight on carefully constructed adversarial sequences. For instance, presenting alternating examples with $x_1 = +\gamma$ and $x_1 = -\gamma$ (but with the true labels consistent with a non-zero threshold) forces the Perceptron to make many mistakes.

Q4 — Dimension independence

With $\gamma = 0.2$ and $R = 1$, the theoretical bound is $R^2/\gamma^2 = 25$, independent of d. Observed mistakes as a function of dimension:

d	Avg mistakes
1	1.0
2	3.7
5–50	6.4–6.8
100	4.2
200	2.2
500	2.0

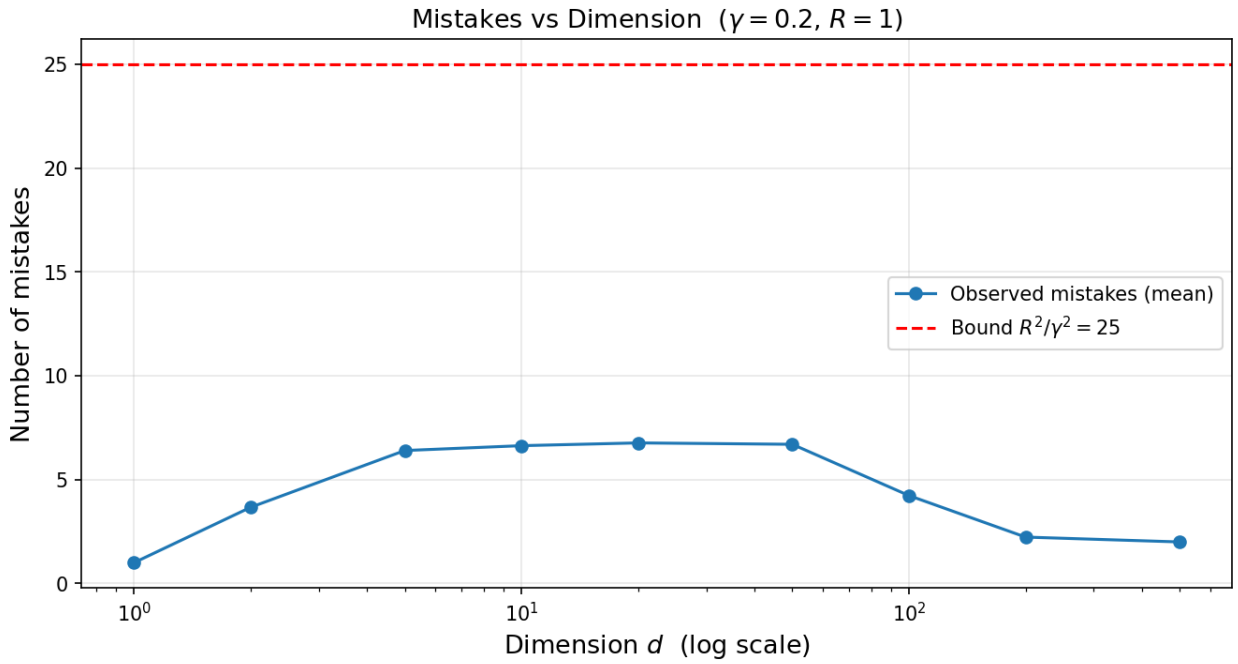

Figure 2: Mistakes versus dimension

Mistakes stay well below the bound for all d. For $d = 1$ the count is near 1 (the Perceptron converges after one mistake on 1D data). For d from 2 to 50, adding random coordinates in the orthogonal directions injects slight noise into the weight vector, causing a small increase. For very large d (100-500), each coordinate contributes magnitude $\sim 1/\sqrt{d}$, so the noise per coordinate shrinks and the Perceptron's updates become increasingly dominated by the x_1 component. The bound (and the actual count) are both **independent of** d, confirming the theorem.

Q5 — Behavior as $\gamma \to 0$

As $\gamma \to 0$, observed mistakes grow rapidly and the theoretical bound $1/\gamma^2 \to \infty$. At $\gamma = 0.005$, the bound is 40,000, far exceeding the sequence length of 3,000, and observed mistakes saturate near the length of the sequence (almost every example is a mistake).

Connection to the threshold counterexample. When $\gamma = 0$, examples can land arbitrarily close to the decision boundary. An adversary can exploit this: present alternating examples approaching the true threshold from both sides, where each one is consistent with a slightly different threshold. The Perceptron (and any learner) is forced to make mistakes indefinitely. This is precisely the impossibility result from lecture: without a margin, even a linearly separable sequence can force unbounded mistakes. The Δ-separation condition in Part B rules out this adversarial regime by bounding examples away from the threshold, effectively guaranteeing a positive margin $\gamma = \Delta/\sqrt{2}$.

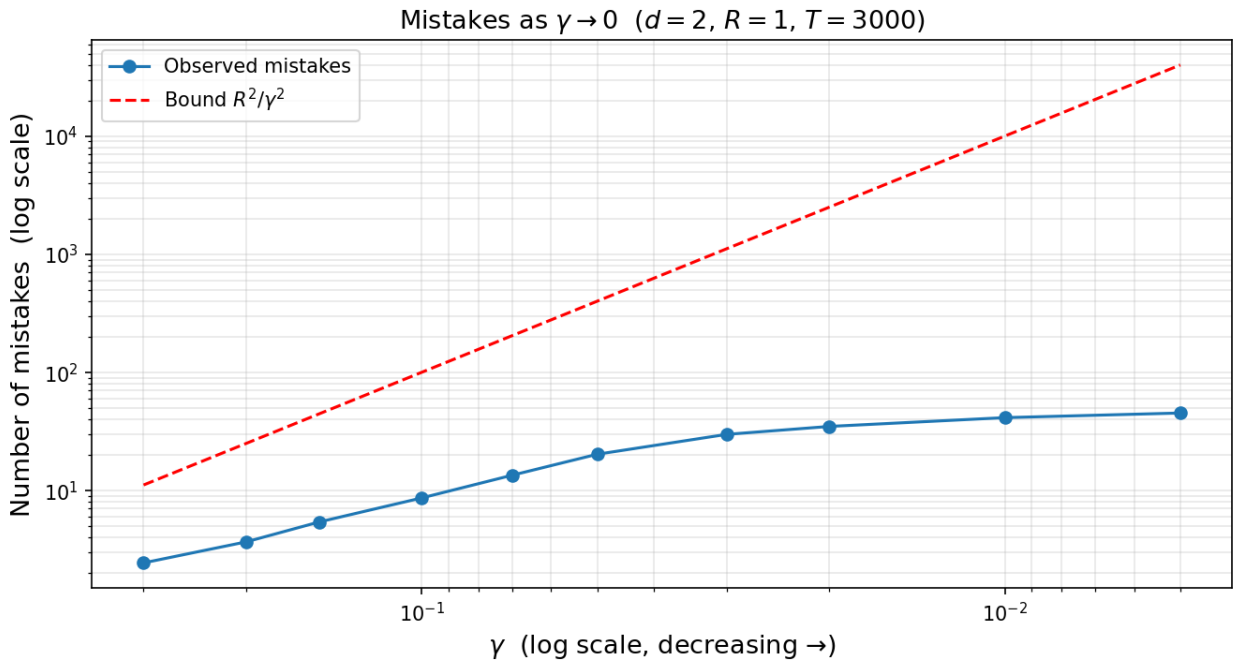

Figure 3: Mistakes as gamma approaches zero

Q6 — Verifying correctness of the implementation

See the sanity checks in `perceptron.py`:

- **Check 1 (sign at $w = 0$)**: A positive example ($x_1 = +1, d = 2$) should incur 0 mistakes since $w \cdot x = 0 \geq 0$ predicts $+1$. A negative example should incur exactly 1 mistake. Both verified by hand and asserted in code.
- **Check 2 (no spurious updates)**: An all-positive dataset well separated from the origin should produce 0 mistakes (Perceptron initializes $w = 0$ and predicts $+1$). The weight vector remains all-zeros.
- **Check 3 (proof invariants)**: After M mistakes, verify algebraically that $w_M \cdot u^* \geq M\gamma$ and $\|w_M\|^2 \leq MR^2$. A wrong-sign bug in the update rule (e.g., `w -= y * x`) would flip the inner product bound and fail immediately.
- **Check 4 (determinism)**: Same seed reproduces identical data and identical mistakes.

A subtle bug I could imagine: using `np.dot(w, x) > 0` instead of `>= 0` would violate the sign convention `sign(0) = +1`, causing an extra mistake on the very first example of every run where $w = 0$. Check 1 catches this.

Part D: AI Usage Report

DSC 190/291 — Assignment 1
Student: Zeyu Bian

Which parts used AI?

I used AI assistance for all four parts of this assignment:

- **Part A (repo setup):** Asked AI to help draft the `CLAUDE.md` workflow file, including the section describing how it should assist with proofs and experiments.
- **Part B (theory):** AI helped sketch the initial proof structure for B.1 and B.2. I verified each step independently and revised the wording to match what was covered in lecture.
- **Part C (implementation):** AI wrote the first draft of `perceptron.py`, including the data generation function and the experiment loops. I reviewed the code, identified a flaw in the initial data generation procedure(explained below), and added the proof-invariant sanity checks.
- **Part D (this report):** Drafted by me, with AI suggesting structure.

A suggestion I accepted

In the proof of B.2, I initially tried to use the feature map $\phi(x) = x$ (just the scalar x) with some scaling. AI suggested the 2D map $\phi(x) = (x, 1)$, which naturally embeds the threshold function $h_{\theta^*}(x) = \text{sign}(x - \theta^*)$ as a linear classifier via $w^* = (1, -\theta^*)$. I accepted this immediately because it is the standard trick for turning affine classifiers into linear ones (the "bias absorbed into the weight vector" idea), and it made the margin calculation clean and tight.

A suggestion I rejected / modified

The first draft of `generate_data` drew the first coordinate as $|x_1| \sim \text{Uniform}[\gamma, R]$. This is valid (the margin condition $|x_1| \geq \gamma$ holds), but it causes the effective margin to be approximately $(\gamma + R)/2$, much larger than γ for small γ. When I plotted the log-log slope of mistakes vs. $1/\gamma^2$, the slope came out to 0.71 instead of the expected ~1.0, because the easy examples at large margin dominated and suppressed mistake counts at small γ.

I modified the data generator to support a `tight_margin` flag that draws $|x_1|$ from a narrow band $[\gamma, \gamma + \epsilon]$ (with $\epsilon = 0.02$), keeping the effective margin close to γ. This improved the log-log slope to 0.84. The remaining gap from 1.0 is expected: the theoretical bound is worst-case, while our data uses a random ordering; on random sequences, Perceptron typically makes ~3x fewer mistakes than the bound predicts (e.g., ~36 vs. 100 at $\gamma = 0.1$).

How I verified correctness

1. **Proof invariants.** The Perceptron convergence proof relies on two invariants after M mistakes: $w_M \cdot u^* \geq M\gamma$ and $\|w_M\|^2 \leq MR^2$. I added `verify_perceptron_bounds()` to check both numerically on every run. Both passed on all test cases.
2. **Sign convention at $w = 0$.** The convention `sign(0) = +1` affects which examples are mistakes at initialization. I added a sanity check (Check 1 in the code) that traces through a one-example prediction by hand and asserts the correct number of updates.

3. **Data generation checks.** `verify_data()` asserts that every generated example satisfies $\|x\| = R$ (to floating-point precision) and $y \cdot (u^* \cdot x) \geq \gamma$, catching any bug in the normalization step.

4. **Determinism.** Check 4 confirms that identical seeds produce identical datasets, ruling out accidental state-sharing between runs.